

Esame Software Engineering (AA 2025/26)

23 Giugno 2026 - Lab. Colossus - Via salaria 113

Enrico Tronci

*Computer Science Department, Sapienza University of Rome
Via Salaria 113 - 00198 Roma - Italy*

tronci@di.uniroma1.it

<https://raise.uniroma1.it>

Esercizio 3 (20 punti)

Un sistema di *e-commerce* vende P prodotti. Il pattern di acquisto del prodotto $i \in \{1, \dots, P\}$ è modellato con il *customer* i . Ci sono quindi P customers, tanti quanti sono i prodotti.

C'è un solo server che gestisce le richieste dei customers e si rifornisce dai fornitori. C'è un fornitore per ogni prodotto. Quindi ci sono P fornitori ed il fornitore i fornisce al server il prodotto i richiesto dal customer i .

Ciascun *customer* può essere modellato come una catena di Markov con un solo stato (0). Il customer i resta nello stato 0 per un tempo τ_i scelto uniformemente a random nell'intervallo $[a1 + b1 * i, a2 + b2 * i]$, dove a, b, c, d sono parametri reali.

Se il customer i decide di fare una transizione (che lo riporta nello stato 0) allora sceglie la quantità $q \in \{1, \dots, Q\}$ del prodotto i uniformemente a random nell'intervallo $[a3 + b3 * i, a4 + b4 * i]$. Questo vuol dire che il customer i ha richiesto al server q items del prodotto i . Si noti che quando il customer permane nello stato 0 nulla viene inviato ai servers.

Il customer invia al server la richiesta (i, q) ed attende dal server la risposta (i, k, v) che definisce quanti items (k) del prodotto i vengono consegnati ed il loro prezzo unitario, cioè per item (v).

Il server ha una variabile **SellBuy** dove memorizza il reward ottenuto dalle vendite ai customers ed il costo sostenuto per acquistare i prodotti dai fornitori. La variabile **SellBuy** è inizializzata a 0.

Il server ha un DB dove sono memorizzate triple del tipo (i, q, v) dove: i è l'indice di un prodotto, q la quantità disponibile del prodotto i , v il costo unitario del prodotto i . Per ciascun prodotto il server può contenere al più W items. Quindi in una tupla (i, q, v) abbiamo sempre $q \leq W$.

Quando il server riceve una richiesta (i, q) dal customer i esegue le seguenti operazioni.

- Determina con il suo DB il numero di items disponibili per il prodotto i . Sia (i, g, v) il contenuto del DB per il prodotto i e sia $k = \min(q, g)$. Il server decrementa di k il numero di items disponibili per il prodotto i , incrementa di $k v$ la sua variabile **SellBuy** di reward e risponde al customer con la tupla (i, k, v) .

Ogni τ secondi il server richiede un prodotto da un fornitore e ricalcola il valore τ . Il valore τ è scelto uniformemente a random nell'intervallo $[\tau_0, \tau_1]$

Il fornitore (e quindi il prodotto) viene scelto uniformemente a random in $\{1, \dots, P\}$ e viene richiesta la massima quantità di prodotto ricevibile dal server, cioè $(W - g_i)$.

Per richiedere q items del prodotto i il server manda il messaggio (q) al fornitore il quale risponde con il messaggio (q, c) dove c è il costo della fornitura.

Alla ricezione del messaggio (q, c) dal fornitore i il server:

- Aggiorna il DB. Quindi la tupla (i, g_i, v_i) diventa $(i, g_i + q, v_i)$.
- Aggiorna la variabile **SellBuy** come segue: **SellBuy** = **SellBuy** - c .

Il fornitore i determina il costo c come segue:

$$c = a5 + b5 * q \quad (1)$$

Cioè il costo della consegna consiste in un costo fisso $a3$ più una parte proporzionale ($b3$) al numero di items inviati.

Sia $\beta(t)$ il valore al tempo t della variabile **SellBuy**. Il *reward rate* $RR(t)$ al tempo t è definito come segue.

$$RR(t) = \frac{\beta(t)}{t} \quad (2)$$

Il reward rate misura la profittabilità del sistema.

Tutte gli scambi di messaggi impiegano tempo r .

Per ogni prodotto i , il DB del server viene inizializzato con la tuple (i, g_i, v_i) dove g_i è scelto uniformemente a random in $\{0, \dots, W\}$ e v_i è scelto uniformemente a random nell'intervallo reale $[b5, b5 * 2]$.

1 Obiettivo

Si vuole riportare in uscita il valore atteso del *reward rate* alla fine della simulazione. Cioè il valore medio del *reward rate* alla fine della simulazione calcolato usando M simulazioni Montecarlo.

2 Parametri

I parametri della simulazione sono:

1. H: orizzonte di simulazione (valore reale positivo)
2. M: numero di simulazioni montecarlo (valore intero positivo)
3. P: numero di prodotti (valore intero positivo)

4. Q: numero di items per prodotto (valore intero)
5. W: max numero di items per prodotto nel server (valore intero positivo)
6. a1, a2, a3, a4, a5: parametri a_1, a_2, a_3, a_4, a_5 del testo (valore reale)
7. b1, b2, b3, b4, b5: parametri b_1, b_2, b_3, b_4, b_5 del testo (valore reale)
8. tau0, tau1: parametri τ_0, τ_1 del testo (valore reale)
9. r: tempo scambio messaggi (valore reale positivo)

3 Formato dei parametri di input

I parametri della simulazione sono forniti nel file `parameters.txt`. Le righe del file `parameters.txt` hanno tutte lo stesso formato:

<ID del parametro> <valore del parametro>

Un esempio di file `parameters.txt` è:

```
H 234.6
M 100
P 10
Q 10
W 100
a1 2.1
a2 2.3
a3 2.4
a4 2.5
a5 2.6
b1 3.1
b2 3.2
b3 3.3
b4 3.4
b5 3.5
r 0.01
```

4 Formato di output

L'output dell'esercizio è memorizzato nel file `results.txt` la cui prima riga è formattata come indicato nelle istruzioni generali.

Le rimanenti righe del file `results.txt` hanno il formato

RR < valore medio reward rate a fine simulazione >

Un esempio di file `results.txt` per il file `parameters.txt` dato sopra è:

2025-01-09-Mario-Rossi-1234567

RR 12.3

5 Struttura Sorgenti

Deve esserci almeno una classe, in un proprio file `cpp`, per ognuno dei processi: `customer`, `server`, `supplier`. Quindi devono esserci almeno i seguenti files: `customer.cpp`, `server.cpp`, `supplier.cpp`.